2026-8-6 Linux信号--信号的捕捉

- 1. 信号捕捉的基本模型
  - 1.1 捕捉是三种递达动作之一
    - 1.1.1 默认、忽略与自定义捕获
    - 1.1.2 捕获与阻塞的区别
    - 1.1.3 捕捉不会创建新进程
- 2. 用户态、内核态与捕捉时机
  - 2.1 为什么不能立刻执行处理函数
  - 2.2 进入内核的三条常见路径
  - 2.3 一次自定义捕捉的四个状态变化
  - 2.4 sigreturn 与现场恢复
- 3. 内核怎样获得控制权
  - 3.1 系统调用主动入内核
  - 3.2 异常与中断被动入内核
  - 3.3 轮询与中断
- 4. 注册信号处理动作
  - 4.1 使用 signal()
  - 4.2 使用 sigaction()
  - 4.3 捕捉时自动屏蔽信号
  - 4.4 推荐的 sigaction 注册方式
- 5. 处理函数中的执行语义
  - 5.1 handler 是异步插入的用户代码
  - 5.2 只调用异步信号安全函数
  - 5.3 volatile sig_atomic_t 与编译器优化
  - 5.4 信号嵌套与处理期间再入
- 6. 系统调用被信号打断
  - 6.1 EINTR 的含义
  - 6.2 SA_RESTART
- 7. 捕捉流程的完整闭环
  - 7.1 从信号到主流程恢复
  - 7.2 关键结论

1. 信号捕捉的基本模型

1.1 捕捉是三种递达动作之一

1.1.1 默认、忽略与自定义捕获

信号具备递达条件以后，内核根据 `handler` 表选择动作：

处理动作	典型设置	执行位置
默认动作	`SIG_DFL`	由内核按信号定义完成，可能终止、暂停或忽略
忽略	`SIG_IGN`	由内核直接丢弃，不进入用户处理函数
自定义捕获	用户提供的 `handler(int)`	切换到用户态执行

“捕获信号”通常指第三种方式。它不是阻止信号产生，也不是阻塞信号，而是在信号到达处理阶段后接管响应动作。

1.1.2 捕获与阻塞的区别

- 阻塞发生在递达之前：信号可以保持未决，暂时不进入处理。
- 捕获发生在递达以后：信号已经通过待处理与阻塞检查，开始执行自定义动作。

因此，一个信号既可以被阻塞，也可以被捕获，但两者位于生命周期的不同阶段。

`SIGKILL`（9）和 `SIGSTOP`（19）是例外：它们既不能被阻塞，也不能被捕获或忽略。

1.1.3 捕捉不会创建新进程

自定义处理函数仍由收到信号的进程自己执行。内核临时改变该进程用户态的执行入口，不会为 handler 新建一个独立进程。

信号是异步事件，handler 可能插入到主控制流的任意可交付位置。程序不能假设它只会在某一条语句之后运行。

2. 用户态、内核态与捕捉时机

2.1 为什么不能立刻执行处理函数

信号产生时，进程可能正在执行关键内核逻辑，或者没有处在可以安全返回用户态的位置。内核会先保存未决状态，在进程即将返回用户态时检查可递达信号。

图示采用经典 32 位 Linux 地址空间模型：

区域	典型地址范围	主要内容
用户空间	`[0, 3 GiB)`	用户代码、栈、堆、数据段
内核空间	`[3 GiB, 4 GiB)`	内核代码与内核数据

64 位系统的地址划分不同，但“用户代码运行在较低权限，内核代码运行在较高权限”的基本边界不变。

用户态不能直接执行内核代码或任意修改内核数据结构；内核态拥有更高权限，也不能直接把用户的 `sighandler` 当作内核代码执行，否则会形成越权入口。

2.2 进入内核的三条常见路径

进程从用户态进入内核态，常见入口有三类：

入口	例子
系统调用	`read`、`write`、`sleep`、`kill`
异常或陷阱	除零、非法内存访问、调试断点
硬件中断	键盘、网络、时钟等设备发出中断

信号检测和自定义捕捉通常发生在内核准备返回用户态之前。不是在信号产生的瞬间强行打断当前内核代码，而是在安全的返回边界决定下一步执行入口。

2.3 一次自定义捕捉的四个状态变化

以主控制流被信号打断为例：

```
用户态执行 main
      |
      | 系统调用 / 异常 / 中断
      v
内核态保存必要上下文
      |
      | 检查 pending、block 和 handler
```

```
      v
用户态执行 sighandler
      |
      | handler 返回，通过 sigreturn 再进内核
      v
内核恢复原上下文
      |
      v
用户态从 main 被中断的位置继续执行
```

这条路径包含四次关键状态变化：

1. 用户主流程进入内核。
2. 内核根据处理动作准备返回用户态，但把返回入口改为 handler。
3. handler 执行完后再次进入内核。
4. 内核恢复原上下文，返回主控制流。

默认动作和忽略动作通常在内核中直接处理完成，不需要额外进入用户态 handler；只有自定义捕获需要多一次用户态执行路径。

2.4 sigreturn 与现场恢复

为了正确返回主控制流，内核必须保存：

- 被信号打断的用户指令位置。
- 用户栈指针和必要的寄存器上下文。
- 信号处理期间的阻塞状态和原信号屏蔽字。
- 返回主控制流所需的其他机器上下文。

自定义 handler 不能像普通函数那样直接 `return` 到内核中的临时调用点。实际运行环境会为它安排一个返回入口；handler 返回后通过 `sigreturn` 类机制再次进入内核，由内核恢复先前保存的上下文。

现代 Linux 通常通过用户态虚拟动态共享对象中的返回跳板完成这一步，不应简单理解为用户每次手工调用一个固定系统调用编号。

3. 内核怎样获得控制权

3.1 系统调用主动入内核

用户程序执行：

```
ssize_t n = read(fd, buf, size);
```

外层库函数最终发起系统调用，CPU 从用户模式切到内核模式。内核完成文件读取或其他工作，在返回用户态前检查当前进程是否已有可递达信号。

如果此时存在自定义捕获信号，原系统调用不一定直接返回主流程，而可能先转到 handler；handler 结束后再恢复原流程。

3.2 异常与中断被动入内核

除零、非法地址、缺页、设备 I/O 完成等事件可能在没有用户主动调用的情况下发生。CPU 或设备通过异常/中断通知内核，内核获得控制权并处理现场。

如果异常与当前进程有关，内核可以把它转换成该进程的信号；当异常或中断处理准备返回用户态时，又会出现信号捕捉检查点。

3.3 轮询与中断

操作系统发现设备事件有两种概念性方案：

- **主动轮询**：内核不断查询设备状态。实现直接，但无效检查很多，效率低。
- **中断驱动**：设备有事件时主动通知 CPU，内核再处理。平时不需要持续查询，效率更高。

键盘输入、网络数据到达等通常采用中断方式。进程阻塞在 `scanf()` 等输入操作时也不需要自己轮询；数据到达后由内核唤醒相关进程。

4. 注册信号处理动作

4.1 使用 signal()

基本接口是：

```
#include <signal.h>
```

```
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

示例：

```
#include <signal.h>
#include <unistd.h>

void handler(int signo)
{
    (void)signo;
}

int main()
{
    signal(SIGINT, handler);

    while (true)
        pause();
}
```

`signal()` 只登记未来的处理动作，不会立即调用 handler。它的可移植历史语义不如 `sigaction()` 清晰，工程代码通常优先使用 `sigaction()`。

4.2 使用 sigaction()

```
#include <signal.h>

int sigaction(int signo,
              const struct sigaction *act,
              struct sigaction *oldact);
```

参数和行为：

- `signo`：要查询或修改的信号编号。
- `act`：新的处理动作；传入空指针时只查询旧动作。
- `oldact`：若非空，保存修改前的处理动作。
- 成功返回 0，失败返回 -1，并设置 `errno`。
  `sigaction()` 可以原子地修改处理动作，并返回旧配置，适合临时安装 handler 后恢复。
  核心字段可以概括为：

```
struct sigaction {
    void (*sa_handler)(int);
    sigset_t sa_mask;
    int sa_flags;

    // 使用 SA_SIGINFO 时，可使用 sa_sigaction 接收附加信息
};
```

4.3 捕捉时自动屏蔽信号

进入 handler 时，内核通常会暂时阻塞当前正在处理的信号，避免相同信号立即再次打断 handler。这个行为由每个信号的处理配置和 `sa_mask` 共同影响。
`sa_mask` 用于声明：

> 执行该 handler 期间，还希望额外暂时阻塞哪些信号。
> 常见 `sa_flags` 如下：

标志	作用
`SA_NODEFER`	处理当前信号期间不自动屏蔽同一信号，允许它再次递达
`SA_SIGINFO`	改用 `sa_sigaction`，接收 `siginfo_t` 和上下文
`SA_RESTART`	对部分被信号打断的系统调用自动重启
`SA_RESETHAND`	处理一次后恢复默认动作，再次到来时不再捕获

信号屏蔽是逐信号控制的，不应把“处理一个信号”理解为整个进程中的所有信号都被阻塞。

4.4 推荐的 sigaction 注册方式

```
#include <signal.h>
#include <unistd.h>

void handler(int signo)
{
    const char message[] = "caught SIGINT\n";
    (void)signo;
    write(STDOUT_FILENO, message, sizeof(message) - 1);
}

int main()
```

```
{
    struct sigaction act {};
    struct sigaction old {};

    act.sa_handler = handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;

    if (sigaction(SIGINT, &act, &old) == -1)
        return 1;

    while (true)
        pause();
}
```

handler 中使用 `write()`，是因为它是异步信号安全函数；`iostream`、`printf`、动态内存分配等接口通常不适合直接在信号处理函数中调用。

5. 处理函数中的执行语义

5.1 handler 是异步插入的用户代码

handler 可能打断任意一段普通用户代码，包括：

- 正在修改一个链表或多个相互关联的变量。
- 正在调用 `malloc`、`free` 等库函数。
- 正在执行标准 I/O 或获取内部锁。
  如果 handler 再次调用与被打断代码共享状态的不可重入函数，可能破坏库内部状态，导致死锁、重复释放或数据损坏。

5.2 只调用异步信号安全函数

信号处理函数应尽量短，并使用异步信号安全函数，例如：

- `write`、`read`。
- `_exit`。
- `kill`、`raise`。
- 简单的赋值和只访问 `volatile sig_atomic_t` 的操作。
  不推荐直接在 handler 中：
- 使用 `malloc`、`free`。

- 使用 `printf`、`iostream`。
- 修改复杂共享数据结构而没有额外同步设计。
  如果 handler 必须通知主流程处理复杂逻辑，可以只设置标志位或写入自管道，由主流程稍后完成真正工作。

5.3 volatile sig_atomic_t 与编译器优化

下面的写法容易产生竞态或优化问题：

```
bool stop = false;

void handler(int)
{
    stop = true;
}

int main()
{
    signal(SIGINT, handler);
    while (!stop)
        ;
}
```

编译器可能认为主循环中 `stop` 不会改变，从而优化循环条件；同时，普通 `bool` 读写也没有表达出与异步 handler 共享的意图。
更适合简单标志的写法是：

```
#include <signal.h>

volatile sig_atomic_t stop = 0;

void handler(int signo)
{
    (void)signo;
    stop = 1;
}
```

`volatile` 要求每次重新读取变量，避免循环被错误优化；`sig_atomic_t` 表示适合在信号处理场景中进行简单原子访问的整数类型。

仅使用 `volatile` 不等于解决所有并发问题。多线程、复杂对象和多个信号共享状态时，仍需要更完整的同步方案。

5.4 信号嵌套与处理期间再入

如果 handler 在执行时同一信号再次产生：

- 默认情况下，同一信号在处理期间通常被暂时阻塞，新的相同标准信号可能只合并成一个未决状态。
- 设置 `SA_NODEFER` 后，同一信号可以再次递达并嵌套进入 handler。
- `sa_mask` 还可以追加其他需要在处理期间暂时阻塞的信号。
  允许嵌套会增加重入复杂度，除非业务确实需要，否则 handler 应尽量保持短小并避免递归式处理。

6. 系统调用被信号打断

6.1 EINTR 的含义

进程阻塞在 `read`、`sleep` 等系统调用时，如果捕获到信号，内核可能先执行 handler，再让原系统调用提前返回。此时调用可能返回 -1，并设置：

```
errno == EINTR
```

这类返回不等于系统调用发生了不可恢复错误，而是表示等待过程被信号打断。调用者可以根据语义决定重试。

6.2 SA_RESTART

安装 handler 时设置 `SA_RESTART`，可以让部分可重启系统调用在 handler 返回后自动继续等待。
但并非所有系统调用都可重启，也可能存在部分数据传输后返回的情况。因此仍应检查返回值、`EINTR` 和实际完成的字节数。

7. 捕捉流程的完整闭环

7.1 从信号到主流程恢复

```
信号产生并保存在 pending
          |
          v
进程进入内核：系统调用 / 异常 / 中断
          |
          v
内核完成当前工作并准备返回用户态
          |
          v
检查未决、阻塞和处理动作
          |
          +-- SIG_DFL / SIG_IGN：内核完成默认或忽略动作
          |
          +-- 自定义 handler
                   |
                   v
          保存上下文并切换到用户态 handler
                   |
                   v
          handler 返回，通过 sigreturn 再进内核
                   |
                   v
          恢复原信号屏蔽字和机器上下文
                   |
                   v
          从主控制流被中断处继续执行
```

7.2 关键结论

- 捕捉发生在递达阶段，不等于阻塞，也不阻止信号产生。
- 自定义 handler 在用户态运行，不会作为内核代码执行。
- 内核通常在返回用户态的安全边界检查并安排信号处理。
- 默认动作和忽略可以完全由内核处理，自定义捕获需要额外进入用户态。
- handler 返回依赖 `sigreturn` 类机制恢复原控制流，而不是回到内核临时栈中的普通调用点。
- handler 具有异步插入性，必须考虑可重入性、异步信号安全和编译器优化。
- `sigaction()` 比传统 `signal()` 更适合精确控制屏蔽字和标志。
- 捕捉后的系统调用可能返回 `EINTR`，必要时重试或使用 `SA_RESTART`。